

JACQUES-Thomas

Students at UNISTRA – *UN*iversité de STRAsbourg

Faculté de physique et ingénierie

## Internship project

### Integration of the MALTA 2 beam test DAQ in Constellation



In the context of the internship made in the ITDC Team

Year of the internship 2026

KEK, Inter-university Research Institute Corporation High Energy Accelerator Research Organization

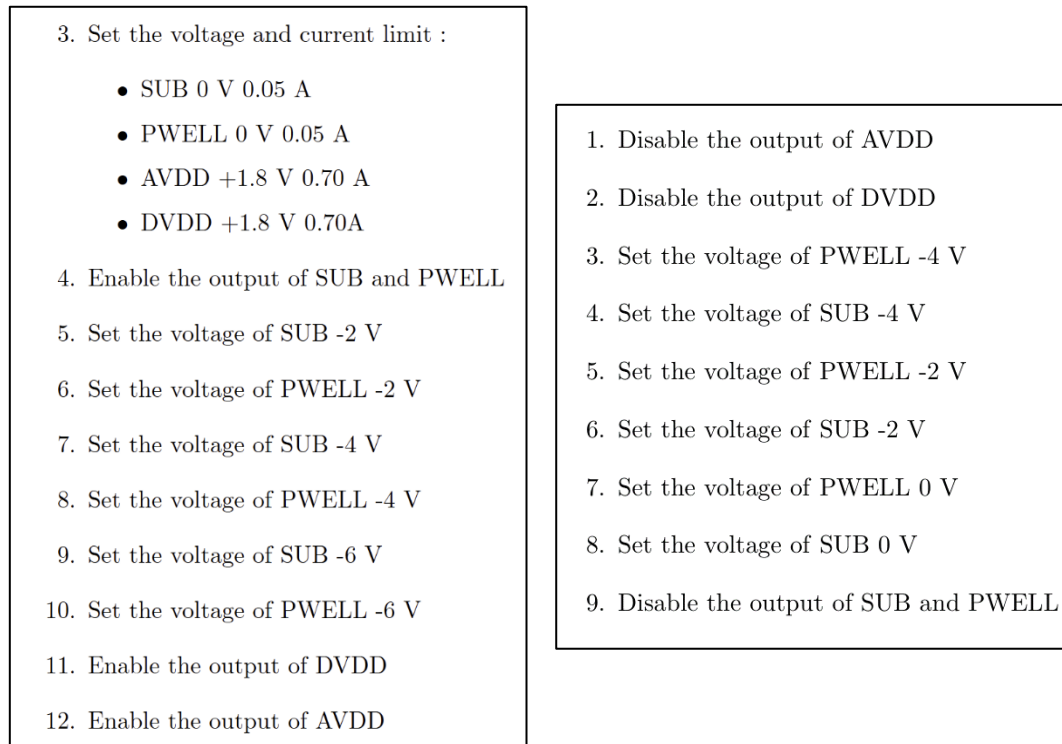


## Table des matières

|                                                                 |    |
|-----------------------------------------------------------------|----|
| Goal of the first project .....                                 | 1  |
| How to control the PowerStation remotely .....                  | 1  |
| 1) Scripts.....                                                 | 1  |
| a) Power On Sequence .....                                      | 2  |
| b) Power Off Sequence .....                                     | 3  |
| c) Monitoring .....                                             | 3  |
| 2) Constellation.....                                           | 3  |
| a) Integrating the powerstation in Constellation.....           | 6  |
| b) Integrating the DAQ in the TLU.....                          | 7  |
| c) Integrating the TLU in Constellation.....                    | 9  |
| Integrating a new satellite (python).....                       | 12 |
| Appendix.....                                                   | 15 |
| Appendix 1 : Description of the TLU satellite architecture..... | 15 |

## Goal of the first project

The goal of the project is to be able to control a [TEXIO PW-A](#) power station remotely to be able to do the power on and off sequence of Malta 2 sensor presented below (as well as to configure the absolute maximum ratings of the powersource) :



**Figure 1: Malta 2 power on Sequence (left) and power off (Right)**

The goal is also to be able to monitor the current consumption of the sensor online (while it is powering MALTA sensors) and to save the data in a logfile. To do that a set of python scripts have been written. Three standalone ones to do the three different actions (power on/off, monitoring) presented [here](#) and a script merging them and integrated in the Constellation framework presented [here](#).

## How to control the PowerStation remotely

### 1) Scripts

The different scripts are located on the MALTA 2 DAQ – *Data AcQuisition System* PC (24B16550, IP address : 130:87:93:197) at the following path :

/home/itdc/work/Thomas/TEXIO\_PWA\_Standalone\_scripts.

There is a READ\_ME file available alongside this document to use the scripts.

To communicate with the powerstation we are using its IF-41USB port connected to the PC via a USB port.

To send and receive messages from the Powerstation we are using the usb.core library. A few low-level functions have been written :

- `send(cmd:str)` : send the command we want to send.
- `recv()` : reads the messages the powerstation is transmitting<sup>1</sup>
- `recv_status()` : reads the status message from the powerstation during the power on sequence. Sometimes the powerstation is not sending the message we requested, so this function ensures we read the message we want
- `empty_buffer()` : empties the message buffer of the powerstation, ensures that there is no shift between the request and the answers

Using those function we are able to control the powerstation remotely with a set of commands given by the powerstation seller, the one used in the different scripts are presented below<sup>2</sup> :

- `SRMODE1` : activates the service request function
- `LC1` : activates local piloting mode
- `SW1/0` : activates/deactivates the main output
- `O(A,B,C,D)0/1` : activates/deactivates an output e.g : `OA1` : output source A activated
- `A(A,B,C,D)XXXX` : configures the maximum current of the output, e.g : `AA0070` → output A maximum current is 700mA
- `V(A,B,C,D)XXXX` : configures the voltage of the output, e.g : `VA0180` → output A potential is 1,8V
- `PR0` : Preset configuration is being applied to the powersupplies<sup>3</sup>
- `DS(1,2,3,4)` : Selects the output to be displayed on the powerstation.
- `ST2/ST4` : request for status messages, the description of them is available in the powersource documentation

## a) Power On Sequence

The powerOn sequence script is entitled : `PowerON_Sequence.py`, to start it, be sure to be in the right folder (`/home/itdc/work/Thomas/TEXIO_PWA_Standalone_scripts` ) and type :

```
sudo python PowerON_Sequence.py
```

After connecting to the powerstation and starting the remote control, This script starts by checking the statuses of the output, if all outputs are off (including the main output) and well configured for the start of the sequence it will skip the initialization. Otherwise it will :

- Turn all output off
- Configure the outputs
- Display all outputs on the powerstation one after the other

---

<sup>1</sup> The Powerstation is only sending messages upon request

<sup>2</sup> Each command must have this architecture : “<[name of the slave powerstation], [request]>”. In our case we are only using one powerstation so the name of It is PW 1

<sup>3</sup> These scripts are only using preset 0 of output (thus the `PR0` command) if you have an application in which it is necessary to have multiple presets for your outputs please check the documentation

After the initialization the power on sequence is running.

At the end of the script we empty the message buffer, we activate the local mode and we close the USB connection.

## b) Power Off Sequence

The powerOff sequence script is entitled : PowerDOWN\_Sequence.py, to start it, be sure to be in the right folder (/home/itdc/work/Thomas/TEXIO\_PWA\_Standalone\_scripts ) and type :

```
sudo python PowerDOWN_Sequence.py
```

After connecting to the powerstation and starting the remote control, this script will run the poweroff sequence, after that we empty the message buffer, we activate the local mode and we close the USB connection.

## c) Monitoring

There are four different monitoring script available for the user, one reads the current and prints them in the cmd, one dumps the values in a logfile, one is tracing curves in real time and the last one, which will be presented into more details below, merges all of them.

The monitoring script is entitled : Monitoring\_complete.py, to start it, be sure to be in the right folder (/home/itdc/work/Thomas/TEXIO\_PWA\_Standalone\_scripts ) and type :

```
sudo python Monitoring_complete.py
```

After the software has been started, a window opens, in which the current curves will be traced. In parallel a logfile named after the date and time of the monitor script launch is created in which the current values will be saved (./logs). When the user wants to stop the software he has to type CTRL+C twice in the cmd.

When the script closes it will empty the message buffer and activate the local mode again before closing the USB connection.

## 2) Constellation

Constellation is a control and data acquisition system for small-scale experiments such as test beams. Each Modules of this framework is autonomous. The online documentation of this framework is available [here](#).

The architecture of this module is presented in the Figure 2:

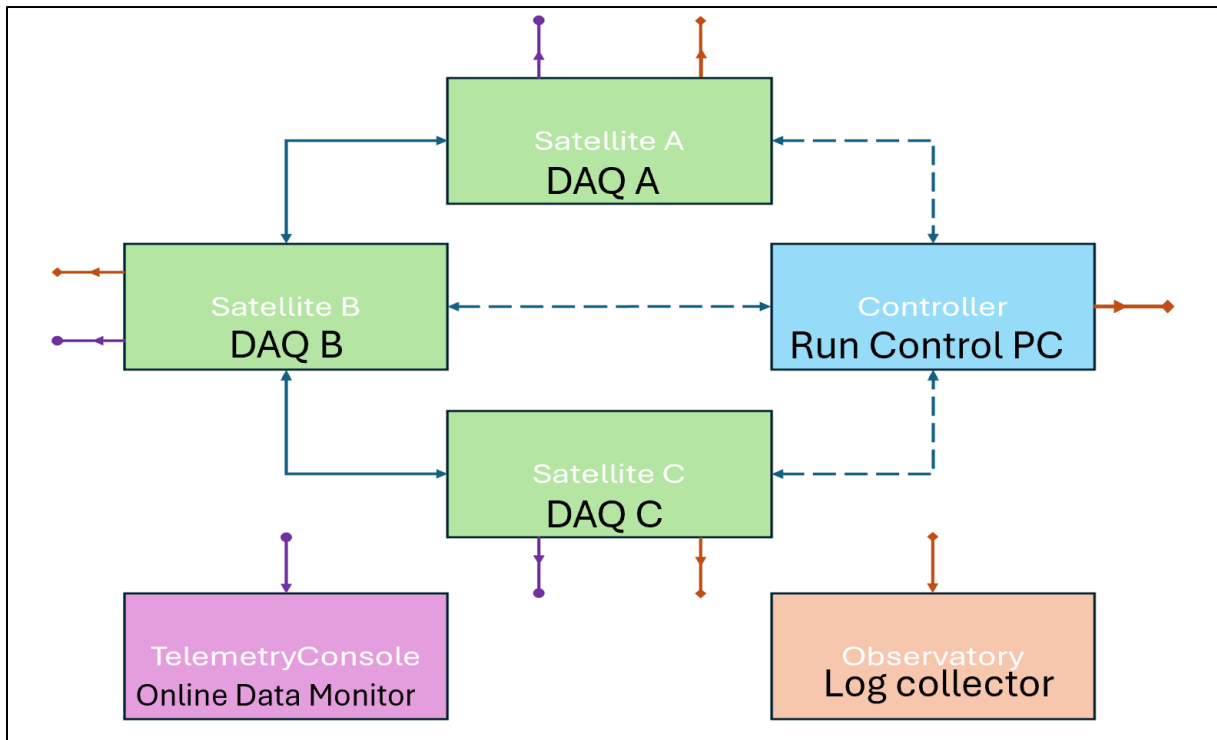


Figure 2: Constellation architecture

As we can see in this figure, the DAQ are named satellites, they are autonomous which means that they do not need a run control PC to supervise them at all time, the satellites are communicating with other modules (other satellites, controller, observatory etc...) by sending heartbeat messages. These messages are sent periodically, they contain the state of the satellite which sent the message the timing before the next heartbeat message and a set of flags presented in constellation's documentation. The state of a satellite is governed by its **fixed** finite state machine presented below :

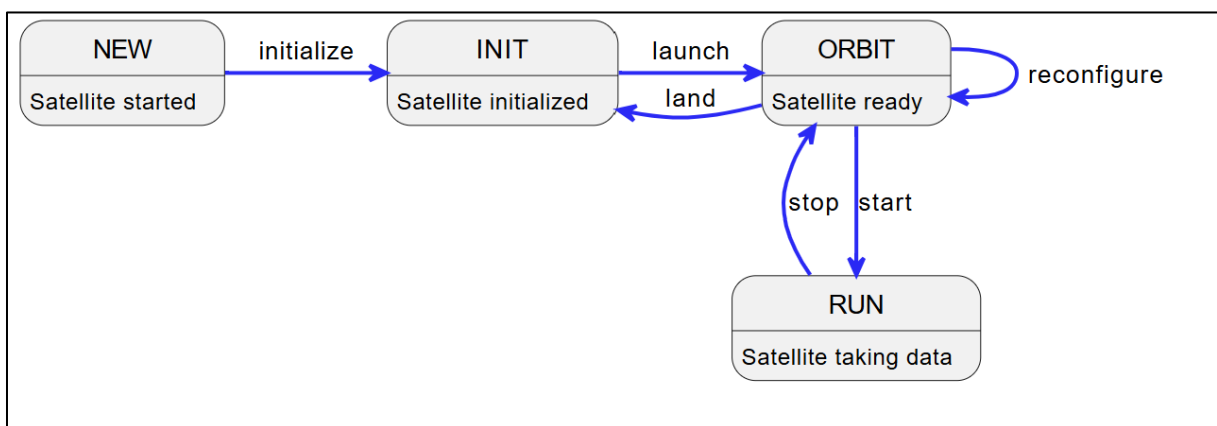


Figure 3: Satellite's FSM

Each transitions (blue arrows) and states (grey box) has to be defined by the user.<sup>4</sup>

<sup>4</sup> The « reconfigure » transition is optional

To start a python satellite (only kind of satellite developed in this project as of this writing ) be sure to be in the right venv (satellites are running on python 3.11) :

```
source /home/itdc/venv/bin/activate
```

And then type :

```
python3 ScriptName.py -n NAME -g GROUP
```

A user can start several times the same satellite in one constellation as long as their names are different, any of those satellites may have different configurations defined in the configuration file.

The other blocks from constellation were downloaded using Flatpak, it means that they are not modifiable and the user should check for any updates of the framework as this one has no stable releases yet.

The controller is the module in charge of piloting the satellites, he is obtaining the state of the satellites and their configuration (if already configured) from the constellation itself. As the satellites are autonomous it means that even if the run control pc crashes it won't have any effect on the constellation. It is also possible to start multiple Controllers in the constellation<sup>5</sup>. The controller has a GUI interface presented below from which we can order satellites to change states.

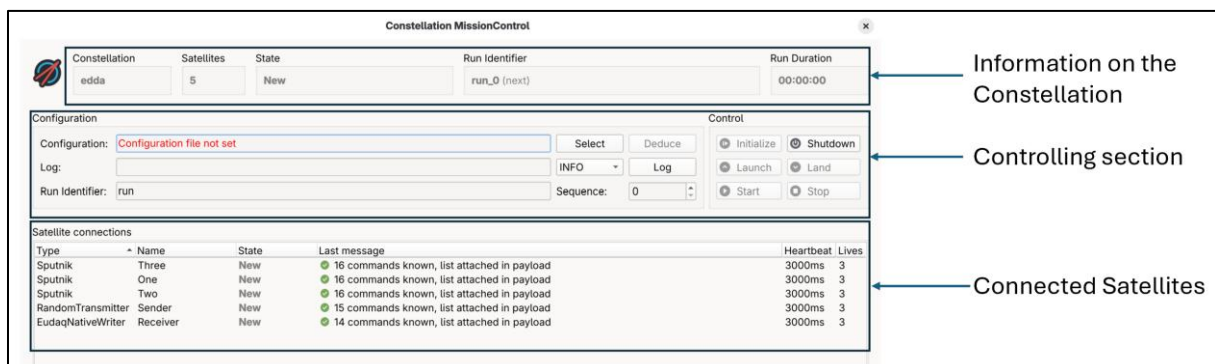


Figure 4: Controller GUI

To start the Controller type (and precise the name of the constellation's group in the popup window) :<sup>6</sup>

```
flatpak run --command=MissionControl de.desy.constellation7
```

The observatory and the telemetryconsole are optional modules, the first is a log collector for all modules of the constellation and the second is an online monitor for the datas of the satellites.

<sup>5</sup> Which will be useful later regarding the reconfiguration of the TLU while it is orbiting.

<sup>6</sup> No need to be in the venv for that, to start the CLI of this modules see the Constellation [documentation](#)

<sup>7</sup> You can also start the Mission control adding the -g GROUP\_NAME at the end of the command so that it connects to it directly (no pop-up window)

To start their GUIs type (and precise the name of the constellation's group in the popup window):

TelemetryConsole :

```
flatpak run --command=TelemetryConsole de.desy.constellation8
```

Observatory :

```
flatpak run --command=Observatory de.desy.constellation9
```

## a) Integrating the powerstation in Constellation

The goal of this project was to take the different scripts described before and merge them together in an only satellite integrated in the constellation framework.

To do that I took a template CLI – *Command Line Interface python based* satellite that I modified to be able to remotely control the powerstation.

The different actions of the powerstation satellite's FSM are described below :

- **initialize (transition):** Connecting to the Powerstation (opening the USB connection)
- **INIT (state) :** No action
- **launch (transition) :** Power on sequence
- **ORBIT (state) :** No action
- **start (transition):** opening log file
- **RUN (state):** Monitoring current (saving on log file + sending data to Telemetryconsole)
- **stop (transition):** closing log file
- **land (transition):** Power off sequence

With this FSM implanted the powerstation can be piloted using constellation's framework.

To start this script first make sure to be in the python 3.11 venv :

```
source /home/itdc/venv/bin/activate
```

Then :

```
cd /home/itdc/work/Thomas/Constellation/template-satellite-python-main/src/TexioPwrStt
```

Finally :

---

<sup>8</sup> You can also start the Telemetry console adding the -g GROUP\_NAME at the end of the command so that it connects to it directly (no pop-up window)

<sup>9</sup> You can also start the Observatory adding the -g GROUP\_NAME at the end of the command so that it connects to it directly (no pop-up window)



```
python3 TexioPwrStt.py -n NAME -g GROUP10
```

The configuration of the powerstation is as followed :

Figure 5: Powerstation configuration

```
[PowerStation.PW1]
PwrSttAdd = "PW 1"
vendor_id = 0x098F
product_id = 0x1001
v_dvdd = 1.8
v_avdd = 1.8
v_pwell = 6.0
v_sub = 6.0
i_dvdd = 0.7
i_avdd = 0.7
i_pwell = 0.05
i_sub = 0.05
short_circuit_threshold_pct = 0.5
```

- PwrSttAdd : Address of the powerstation, if the address does not correspond to any connected device the satellite will be put in error state during initialization
- v\_dvdd/avdd/pwell/sub : Value for the voltage of all different outputs.
- i\_dvdd/avdd/pwell/sub : Value for the current of all different outputs.
- Short\_circuit\_threshold\_pct : if the Tension of one output is below [Short\_circuit\_threshold\_pct \* v\_dvdd/avdd/pwell/sub] during a run, then the powerstation will be put in error state<sup>11</sup>

If an error is occurring during the remote control, if possible, the satellite performs the power down sequence, the local mode is reactivated and the usb connection is closed. The user has to do an initialization again.

If a short circuit is detected it is considered as an error but it takes some time for the satellite to detect the actual short circuit...

## b) Integrating the DAQ in the TLU

The goal of this integration is to start the DAQ script and the online monitor using a satellite, it means that the data from the DAQ are not sent in the constellation.

Before starting the satellite **BE SURE** to do the setup of the DAQ files :

```
source /home/MaltaSW/setup.sh
```

To start the satellite be sure to be in the python 3.11 venv and in the right directory (/home/itdc/work/Thomas/Constellation/MaltaDAQ/src/MaltaDAQ) and then type :

```
python3 MaltaDAQ.py -n NAME -g GROUP12
```

The other way to do those three previous step (setup.sh, starting the venv, starting the satellite) is to run the script located here : /home/itdc/work/Thomas/Constellation/MaltaDAQ

```
source START_DAQ_SHELL.sh
```

<sup>10</sup> Be sure to put the same NAME as the one in the configuration file.

<sup>11</sup> If the Current of an output is superior to  $0.9 * i\_dvdd/avdd/pwell/sub$  then a warning is sent to the observatory

<sup>12</sup> Be sure to put the same NAME as the one in the configuration file.

The different actions of the DAQ satellite's FSM are described below :

- **initialize** : Getting the argument of the DAQ and Online Monitor script
- **INIT** : None
- **launch** : None
- **ORBIT** : None
- **start** : Starting the DAQ C++ script, creating a repertory for the data that has the same name as the run
- **RUN** : Starting the OnlineMonitor to display histograms
- **stop** : Closing OnlineMonitor and the DAQ script, this transition can only happen after the TLU satellite has been stopped (if a TLU satellite exists in the constellation)
- **land** : None

The configuration of the DAQ satellite is presented below :

```
[MaltaDAQ.NAME]
binary_path = "/home/MaltaSW/build/MaltaDAQ/MaltaMultiDAQ"
daq_config =  "/home/MaltaSW/MaltaDAQ/configs/2DUT_Const_Test.txt"
work_dir = "/home/MaltaSW/MaltaDAQ"
monitor_dir = "/home/MaltaSW/ReadyJuneBeamtest/config/2malta_dut"
monitor_script = "run_onlinemonitor.sh"
conditions.require_stopping_after = ["MaltaTLU.oui"]
```

Figure 6: Configuration of the DAQ satellite

- **binary\_path** : path to the DAQ script binary
- **daq\_config** : path to the DAQ configuration
- **work\_dir** : path to the workdir
- **monitor\_dir** : path to the shell starting the online monitor
- **monitor\_script** : name of the shell starting the monitor script<sup>13</sup>
- **condition.require\_stopping\_after** = ["MaltaTLU.oui"] : The DAQ will stop its run ONLY after MaltaTLU.oui stopped it. If there is no TLU satellite named this way in the constellation you need to remove this condition<sup>14</sup>

The output path for the data is created according to the name of the run.

e.g : as of this writing, if the current run is named run\_81, the datas will be saved at the following path

/home/itdc/work/Thomas/Constellation/MaltaDAQ/Data\_test/run and the run number in the rootfile made by the DAQ script will be 81

---

<sup>13</sup> The shell starting the online monitor is to be changed a bit, at the end of the shell the user should change :

`${CORRY_EXEC} -c "${TMP_MONITOR}"` to `exec ${CORRY_EXEC} -c "${TMP_MONITOR}"` so that the online monitor can be closed using the satellite at the end of a run.

<sup>14</sup> The complete description of those transition is available [here](#)

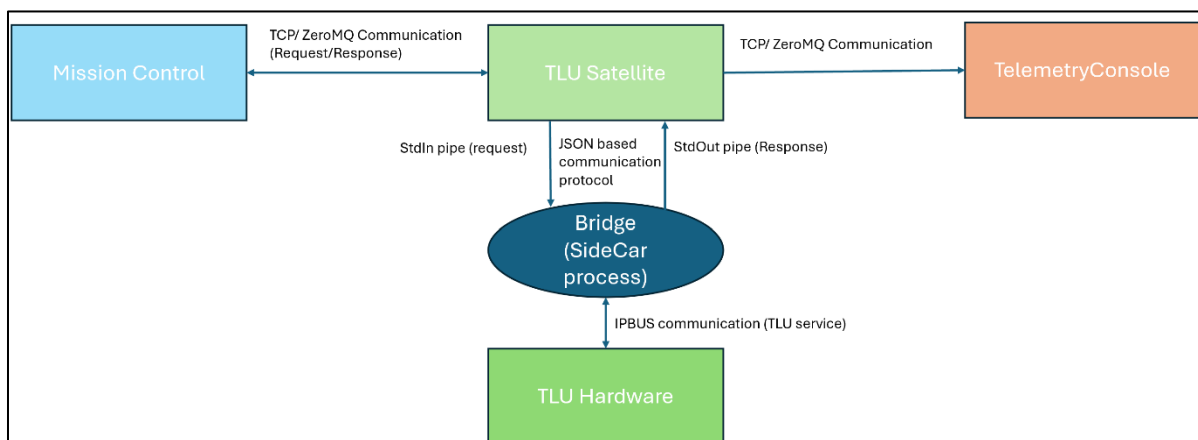
The user still need to ensure that the configuration file for the DAQ is suitable for its experiment (number of Readout boards, name and configuration file of each of them) as well as the starting shell of the online monitor and its configuration (specially the path to the DATA).

The current configuration files and starting shell of the online monitor can be found here :  
/home/MaltaSW/ReadyJuneBeamtest/config

### c) Integrating the TLU in Constellation

There was no premade Satellite for the Malta TLU (There is one for the TLU AIDA 2020 if needed one day). So the goal was to integrate the CLI version of the TLU in the constellation framework.

The TLU scripts (and low-level drivers) are running using python 3.9 but the constellation ones are running on python 3.11. The satellite is controlling the TLU through a bridge subprocess rather than importing the driver directly. The architecture of this communication is presented in Figure 7.



**Figure 7: Architecture for controlling the TLU**

The TLU satellite is working as a regular satellite for the user.<sup>15</sup>

To start it be sure to be in the python 3.11 venv and in the right directory (/home/itdc/work/Thomas/Constellation/TLU\_MALTA/src/TLU\_MALTA) and then type :

```
python3 TLU_MALTA.py -n NAME -g GROUP16
```

<sup>15</sup> If one wants to understand in more details how this architecture is working check [Appendix 1 : Description of the TLU satellite architecture](#)

<sup>16</sup> Be sure to put the same NAME as the one in the configuration file

The different actions of the TLU satellite's FSM are described below :

- **initialize (transition):** Bridge connection + configuration file parsing
- **INIT (state):** None
- **launch (transition):** Configuration of the TLU
- **ORBIT (state):** None
- **start (transition) :** Resetting TLU counters, starting it. This transition can only happen if the DAQ satellite is running.
- **RUN (state):** Monitoring counters (sending data to Telemetryconsole)
- **stop (transition):** Opening a log file, writing a summary of the run (configuration, counter values) closing log file, resetting counters, stopping TLU
- **land (transition):** Bridge disconnection + closing it
- **reconfigure (transition) :** ONLY usable using the CLI controller, to see how to use it see the example below : 18.

As of this writing the configuration of the TLU is written in a TOML and is declared as follows :

```
[MALTA_TLU_NAME]
bridge_python = "/home/atlas/sw/lcg/releases/LCG_104d/Python/3.9.12/x86_64-el9-gcc13-opt/bin/python3"
bridge_script = "/home/itdc/work/Thomas/Constellation/Bridge.py"
uri = "192.168.200.20"
address_table = "/home/MaltaSW/MaltaTLU_AlinxSW/configs/tlu_addresses.xml"
bridge_repo_root = "/home/MaltaSW/MaltaTLU_AlinxSW/"
bridge_pythonpath = "/home/MaltaSW/installed/x86_64-el9-gcc13-opt/lib:/home/MaltaSW/installed/share/lib/python"
bridge_ld_library_path = "/home/atlas/sw/lcg/releases/gcc/13.1.0-b3d18/x86_64-el9/lib64:/home/atlas/sw/lcg/releases/gcc/13.1.0-b3d18/x86_64-el9/lib:/home/MaltaSW/installed/x86_64-el9-gcc13-opt/lib"
sc_enabled = false
plane_1 = true
plane_2 = true
plane_3 = false
plane_4 = false
plane_5 = false
plane_6 = false

veto_sc = 1
veto_1 = 1
veto_2 = 1
veto_3 = 1
veto_4 = 1
veto_5 = 1
veto_6 = 1

max_rate_hz = 0
max_rate_enabled = true
LIA = 1000

width_sc = 80
width_1 = 80
width_2 = 80
width_3 = 80
width_4 = 80
width_5 = 80
width_6 = 80
width_LIA = 100

conditions.require_starting_after = ["MaltaDAQ_gui"]
```

Figure 8: Part of the MALTA TLU configuration file for Constellation

- **bridge\_python:** Path to the Python 3.9.12 interpreter, provided by the ATLAS/LCG software stack (release LCG\_104d). Used to launch the `tlu_bridge.py` subprocess, since the low-level TLU driver (Herakles) only works under this specific Python version.
- **bridge\_script:** Path to the `tlu_bridge.py` script itself. This is the bridge subprocess: it imports Herakles / TLUService directly and exposes a simple JSON-lines request/response protocol over stdin/stdout to the satellite.
- **Uri:** Network address (IP) of the TLU hardware.
- **address\_table:** Path to the XML file defining the TLU's register map (register names and their corresponding addresses).
- **bridge\_repo\_root:** Root directory of the TLU software repository (the folder containing `gui/` and `lib_TLU/`). Added to the bridge subprocess's `sys.path` so it can import `gui.tlu_service` (and, through it, the TLU driver).
- **bridge\_pythonpath:** Extra `PYTHONPATH` entries required by the bridge subprocess to locate the Herakles module and its supporting Python libraries (from the ATLAS/LCG install). Set

explicitly so the bridge does not depend on whatever shell environment happened to launch the satellite (MissionControl, a systemd unit, etc.).

- `bridge_ld_library_path`: Extra `LD_LIBRARY_PATH` entries required by the bridge subprocess so the dynamic linker can find the correct shared libraries (e.g. a newer `libstdc++`) needed by Herakles.so. Without this, importing Herakles can fail with a library version mismatch even when `PYTHONPATH` is otherwise correct.
- `sc_enabled`: scintillator
- `plane_1/2/3/4/5/6`: Malta TLU planes, if a Malta 2 sensor is connected to this plane → set it active.
- `veto_sc`: scintillator veto
- `veto_1/2/3/4/5/6`: veto planes
- `max_rate_hz`: maximal rate of the trigger sent by the TLU, the system works with a radiation source (high rate) with no need of a specified `maximal_rate`
- `max_rate_enabled`: activate the maximal rate, if `max_rate_hz` is different than 0, this parameter is automatically set to true
- `L1A`: level one trigger accept for atlas experiment
- `Width_sc`: duration between two trigger on scintillator veto
- `Width_1/2/3/4/5/6`: duration between two trigger on planes veto
- `Width_L1A`: duration between two trigger on level one trigger veto
- `Condition.require_starting_after = ["MaltaDAQ.oui"]`: The TLU will only start its run after the MaltaDAQ.oui satellite has started it, if there is no DAQ in the constellation you need to remove this condition.<sup>17</sup>

Remark : As of this writing :

- using the GUI MissionControl it is not possible to reconfigure the TLU while it is orbiting (Constellation version 0.8.1 and older). The configuration file is only parsed during the initialization if the user wants to change the configuration of the TLU after a run he should make it land and redo an initialization.
- Using the CLI MissionControl it is possible to reconfigure the TLU.<sup>18</sup> To start the CLI Mission Control be sure to be in the right venv and then type :

```
Controller -g GROUP
```

An IPython Command line interface should be running.

To reconfigure the TLU using the CLI MissionControl follow the example below :

If the user wants to activate the plane 2 and deactivate the plane 1 he should type the following script in the CMD :

```
Recfg = {"plane_2": bool(1), "plane_1": bool(0)}  
ctrl.constellation.MaltaTLU.NAME.reconfigure(recfg)
```

---

<sup>17</sup> The description of those transitional state is available [here](#).

<sup>18</sup> It is possible to run several instances of MissionControl at the same time which means that it is possible to run the GUI version (easier to use) and the CLI (just useful for the reconfiguration) version simultaneously.

Then the Planes would be configure as wanted and the user is able to start the run using either the GUI or the CLI Mission Control without having to re-initialize the TLU.<sup>19</sup>

## Integrating a new satellite (python)<sup>20</sup>

This chapter will present the different steps to add a new satellite in constellation's framework.

It will be considered that Constellation is already downloaded on the PC, if not, follow the instructions [here](#).<sup>21</sup>

The fastest way to integrate a new satellite in the framework is to use the satellites template [here](#).

Than follow these steps :

1. Unzip the project
2. Rename the template using the rename\_template script :
  - a. rename-template.py MyName
3. open the python script that has the same name you just defined → cd src/MyName/MyName.py
4. I was personally merging the \_\_main\_\_.py and the MyName.py (found it easier to have all in one script)
5. The python file should look like this :

---

<sup>19</sup> If one wants to check if the configuration has been made correctly, it is possible by using the “deduce” method from the Controller. It will ask you to save the deduced configuration somewhere and then you will be able to check it.

<sup>20</sup> The only satellite that we are using right now are python satellites, if one wants to integrate a C++ satellite, he should follow this [instructions](#).

<sup>21</sup> The flatpak download is not modifiable but is useful because it contains the different missioncontrol, observatory and telemetry console GUIs. If one wants to do a hand on exercise of the framework he can follow the tutorial also available at the link above.

```

"""
SPDX-FileCopyrightText: 2025 DESY and the Constellation authors
SPDX-License-Identifier: EUPL-1.2
"""

from constellation.core.satellite import Satellite, SatelliteArgumentParser
from constellation.core.logging import setup_cli_logging
from constellation.core.configuration import Configuration

class MyName(Satellite):
    pass

def main(args=None):
    """Template satellite"""

    # Get a dict of the parsed arguments
    parser = SatelliteArgumentParser(description=main.__doc__)
    args = vars(parser.parse_args(args))

    # Set up logging
    setup_cli_logging(args.pop("level"))

    # Start satellite with remaining args
    s = MyName(**args)
    s.run_satellite()

if __name__ == "__main__":
    main()

```

Figure 9: template satellite

Then one should add all the different method of a satellite in his satellite's class :

- Def `do_initializing(self, config: Configuration)` : The configuration file is only parsed in the initializing method, you should register the metrics for the telemetry console in this method
- Def `do_launching(self)` : Method to launch the satellite in orbit (state waiting for a run to start), you can configure your satellite (and the hardware it is piloting) in this method.
- Def `do_starting(self, run_identifier : str)` Starting a run, the run identifier comes from the mission control
- Def `do_stopping(self)` : stop the run
- Def `do_landing` : make the satellite land → back to initialized state
- Def `do_run(self)` : method running in a loop while the run is started :

```
def do_run(self) → str:
    # Check if a stop of the run was request
    while not self.stop_requested():
        # Do work
        ...
    return "Finished acquisition."
```

Figure 10: Example of run method implementation<sup>22</sup>

There is also a set of optional methods :

- Def do\_interrupting : never used it, but apparently it is the transition to the safe state see [here](#) for more details.
- Def do\_reconfigure(self, partial\_config : Configuration) : this method can only be called using the CLI Controller, it takes a dictionary of the key one wants to reconfigure as a parameter.

If an error is occurring one should raise a RuntimeError because all methods are wrapped in a try/except. By raising a RuntimeError the satellite will be put in error state, and run the failgracefully method in which the user should define a behaviour to stop the satellite safely.  
e.g

If you have any questions,

Contacts :

- Contact for Constellation developpers and maintainers :
  - Simon Spannagel, DESY, [simon.spannagel@desy.de](mailto:simon.spannagel@desy.de)
  - Stephan Lachnit, DESY, [stephan.lachnit@desy.de](mailto:stephan.lachnit@desy.de)
- Software developer for the first MALTA 2 test bench integration :
  - Thomas JACQUES, CNRS, [thomas.jacques@etu.unistra.fr](mailto:thomas.jacques@etu.unistra.fr)

---

<sup>22</sup> The -> str indicates that this method is returning a str when it ends

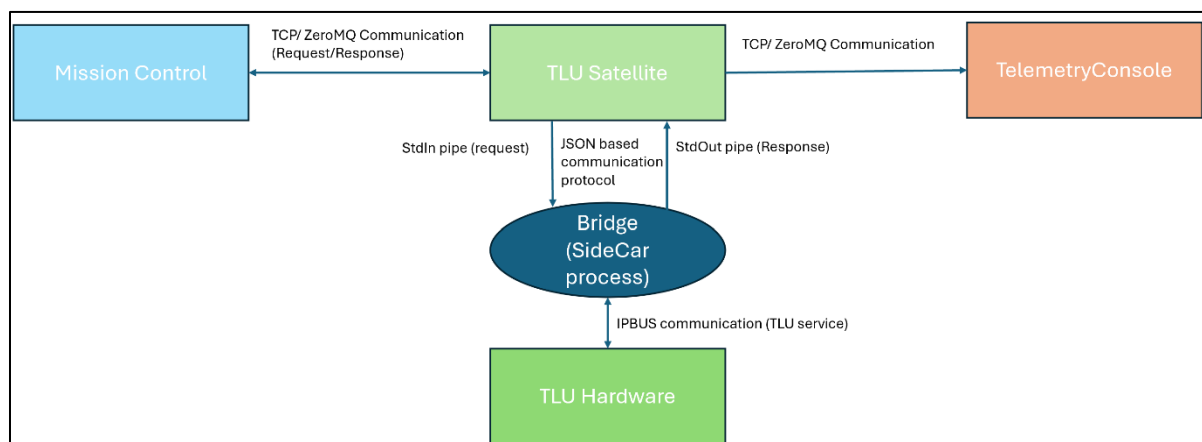


## Appendix

### Appendix 1 : Description of the TLU satellite architecture

As explained upper there is a problem between the python version of the TLU softwares and compiled files (3.9) and the one from constellation (3.11)

It has been chosen to design a sidecar type architecture to resolve this problem. The architecture is reminded below :



**Figure 11: Reminding of the Architecture for the TLU satellite**

Description of the bridge :

The bridge is the block that is communicating with the TLU hardware, the TLU satellite itself does not communicate at all with it. Which means that the TLUService object coming from the tlu\_service.py file describing all the low and high levels communication method with the TLU is instantiated in the TLU bridge. The communication between the Satellite and the Bridge is done via a simple JSON-lines request/response protocol over stdin/stdout. The action for each request of the satellite are described in Bridge.py. This script is started automatically when initializing the satellite. The TLUBridgeClient class is described in TLU\_MALTA.py it is describing the method to start the bridge, and the low and high levels method to communicate with it/control the TLU.

Description of the TLU satellite :

As described before the TLU satellite is a regular satellite for constellation's framework. It incorporates the definition of the bridge's class, the communication with it (sending JSON request/reading JSON messages) and all the methods a satellite usually has.

